Amazon SDE2 Interview Prep

Part 1: STAR answers for all 16 Leadership Principles
Part 2: Full story bank, grouped by Slice and StonksAI

Roshan Jha · Slice (Software Developer) · StonksAI (Co-founder, backend built 0 → 1)

How to use this document

• **[Brackets] are details only you know.** Replace each with what actually happened, or drop the story. Interviewers probe every claim, and invented specifics fall apart under follow-ups.

• **Principles 6 and 15 are templates.** Your resume has no mentoring or team-culture example, so you need real ones.

• **Your arc for SDE2:** at Slice you learned to build production systems at scale inside a team; at StonksAI you owned everything 0 → 1. Together: you can execute with rigor *and* own ambiguous problems end to end.

• **Don't tell the same story twice in one loop.** Interviewers compare notes afterward.

• **Aim for about 2 minutes** on the first telling, then let follow-ups pull detail. Say "I" for your actions and be precise about what teammates did.

Story map: which story answers which principle

#	Leadership Principle	Story	Source
1	Customer Obsession	NPS platform	Slice
2	Ownership	SLO dashboards	Slice
3	Invent and Simplify	MQTT	Slice
4	Are Right, A Lot	Architecture with no data	StonksAI
5	Learn and Be Curious	SDE1 to co-founder	Both
6	Hire and Develop the Best	TEMPLATE: needs real example	Your choice
7	Insist on the Highest Standards	Referral product	Slice
8	Think Big	Experimentation platform	Slice
9	Bias for Action	UPI Rewards	Slice
10	Frugality	Bootstrapped budget	StonksAI
11	Earn Trust	Bad technical decision	StonksAI
12	Dive Deep	Redis and p99	Slice
13	Have Backbone; Disagree and Commit	Co-founder disagreement	StonksAI

#	Leadership Principle	Story	Source
14	Deliver Results	Launch deadline	StonksAI
15	Strive to Be Earth’s Best Employer	TEMPLATE: needs real example	Your choice
16	Success and Scale Bring Broad Responsibility	Referral fraud and honest users	Slice

Part 1: Answers by Leadership Principle

One STAR answer for each of the 16 principles, with likely follow-ups.

1. Customer Obsession

Story: **NPS platform** · Source: Slice

> *Sample question: “Tell me about a time you went above and beyond for a customer.”*

Situation

At Slice, our Google Play rating was 3.3. For a fintech app, that’s a real problem, because people decide whether to trust you with their money partly from that number. We also didn’t know *why* it was low. Feedback came in through scattered channels like support tickets, random reviews and social media, and the team was mostly guessing what users disliked.

Task

I wanted to replace guessing with a reliable system: hear from customers at the right moment, understand what was hurting them, and turn that into fixes. I owned it across both the backend and the Android app.

Action

I built an end-to-end NPS platform, and I spent most of my design effort on three things.

Asking at the right moment. A survey that interrupts someone mid-payment is itself a bad experience. So the backend decided who to ask and when, [after a completed transaction, not too often, with a cooldown], so we got honest answers without annoying people.

Routing responses so we could act. Users who rated us highly were nudged toward leaving a Play Store review. Users who were unhappy were routed to in-app feedback, where their issue could actually reach us and be resolved. An unhappy user posting a 1-star review gets no help, while one who reaches us can.

Closing the loop. I aggregated responses into themes, [e.g., confusion in a specific flow, slow screens, a recurring bug], and brought them to the team so priorities followed real customer pain instead of assumptions.

Result

The Play Store rating rose from **3.3 to 4.5**. Part came from asking happy users at better moments, and part came from the fixes the feedback revealed. Both mattered.

Lesson

Collecting feedback isn’t customer obsession. Acting on it is. Today, I’d also track how quickly feedback themes turn into shipped fixes.

Likely follow-ups

Q: Isn’t routing unhappy users away from the Play Store manipulative?

Nobody was blocked from reviewing. We gave unhappy users a faster path to an actual fix, and the rating rose partly because we fixed what they reported.

Q: How did you avoid survey fatigue?

[Frequency caps and cooldowns per user.]

Q: What was the most surprising thing customers said?

[Have one real example ready.]

2. Ownership

Story: **SLO dashboards and alerting** · Source: Slice

> *Sample question: “Tell me about a time you took on something significant outside your area of responsibility.”*

Situation

At Slice, our critical services had no shared definition of ‘healthy.’ We didn’t have agreed latency or availability targets, and alerting was patchy. Too often, we found out about problems from support tickets or users before our own systems told us. For a payments product, that’s a trust problem.

Task

Reliability tooling wasn’t assigned to me; my job was building product features. But I was regularly debugging issues that monitoring should have caught earlier, and nobody owned this gap, so I took it.

Action

Defining what good means. For each critical service, I defined SLOs for latency, availability and error rates, based on [historical data and what users actually experienced], not arbitrary numbers.

Making it visible. I built Grafana dashboards around those SLOs, so anyone on the team could see a service’s health at a glance.

Alerting people would trust. If alerts fire constantly, people ignore them; if thresholds are too loose, you miss real incidents. I tuned thresholds against historical data and made sure each alert pointed to a specific action.

I did this alongside my feature work, and walked the team through the dashboards so it became shared, not just mine.

Result

The team began catching incidents faster, often before users noticed, and reliability improved. [Give one concrete example: an alert caught X before it reached users.] Conversations changed from ‘it feels slow today’ to ‘we’re burning our latency budget on this endpoint,’ which made prioritization easier.

Lesson

If I see a gap that hurts customers and nobody owns it, I treat it as mine. Next time I’d bring the team in earlier, so ownership spreads faster.

Likely follow-ups

Q: How did you pick the SLO targets?
[Historical percentiles plus user impact.]

Q: How did you avoid alert fatigue?
[Actionable alerts only, tuned thresholds, regular review of noisy alerts.]

Q: How did your manager react to you doing this outside your scope?
[Real answer.]

> Backup story: the StonksAI production outage, where I was the entire on-call, restored service, root-caused it and added monitoring.

3. Invent and Simplify

Story: **MQTT replacing polling** · Source: Slice

> *Sample question: “Give me an example of a complex problem you solved with a simple solution.”*

Situation

Our Android app at Slice repeatedly polled backend APIs to check whether something had changed, such as [transaction or reward status]. When I looked at the traffic, the large majority of those requests returned ‘no change.’ We were spending server capacity on empty calls, and users were spending battery and mobile data on them.

Task

Cut that wasted load without making the app feel staler, and ideally make it more real-time.

Action

I started by **measuring**, confirming from request logs how much polling traffic was redundant, so I knew the size of the win before changing anything.

Then I **compared options**:

- Longer polling intervals were the simplest, but would make data staler.
- WebSockets would work, but persistent connections at mobile scale are heavier on servers and device battery.
- MQTT is a lightweight publish-subscribe protocol built for unreliable networks, which fits mobile users.

I chose MQTT. The server publishes only when state actually changes, and the client subscribes. ‘Keep asking’ became ‘get told when something happens.’

I **managed the risk**: gradual rollout behind a feature flag with polling as fallback, so a broker problem wouldn’t break the feature, plus reconnect handling so a phone that went offline fetched any missed state when it came back.

Result

API calls dropped by about **90%**, and server load dropped with them. Users got updates in real time instead of waiting for the next poll. The simpler model was both cheaper and better.

Lesson

The simplest fix isn’t always the first one you think of. Changing the communication model removed the problem instead of tuning around it.

Likely follow-ups

Q: Why not WebSockets or server-sent events?
Heavier connection management on mobile. MQTT is designed for flaky networks and low overhead.

Q: What if a client misses a message?
On reconnect, the client fetched current state, so correctness never depended on every message arriving.

Q: How did you measure 90%?
[Calls per active user, same time window, before vs. after rollout.]

4. Are Right, A Lot

Story: **Architecture with no data** · Source: StonksAI

> *Sample question: “Tell me about a strategic decision you made without clear data or benchmarks.”*

Situation

When I co-founded StonksAI, I built the backend from scratch, starting with an empty repository. We had no users, no traffic data and no benchmarks. Every architecture decision was a bet, and as the person building the backend, they were mine to make.

Task

Choose an architecture that let us ship quickly without boxing us into a rewrite once real users arrived.

Action

I separated reversible decisions from irreversible ones. Choosing a framework or organizing internal code is cheap to change later. The data model, public API contracts and authentication are expensive once users and clients depend on them. So I moved fast on the cheap decisions and spent real design time on the expensive ones.

I chose the operating model for a tiny team. I compared [microservices with a modular monolith]. Microservices give independent scaling, but bring deployment, networking and monitoring overhead that would have consumed my time. I chose [a modular monolith] with clean internal boundaries, so splitting out a service later would be a refactor, not a rewrite.

I tested the riskiest assumption first, [e.g., whether the core flow could handle realistic load], with a quick prototype before committing.

I sought disagreement. I walked [experienced engineers / mentors] through the design and asked them to find holes. [One thing they changed my mind on.]

Result

We shipped the first version in [X weeks]. [What held up, e.g., ‘the core data model never needed a painful migration.’] Not everything was right: [one thing I’d change, e.g., ‘I drew one module boundary in the wrong place and had to refactor it.’]

Lesson

When data is missing, I make the irreversible decisions carefully and keep everything else cheap to change. Being right a lot partly means being easy to correct when you’re wrong.

Likely follow-ups

Q: What would make you split the monolith?
[A component with very different scaling needs, or team growth.]

Q: What did you get wrong?
[Have a real answer.]

5. Learn and Be Curious

Story: **From SDE1 to co-founder** · Source: Slice + StonksAI

> *Sample question: “Tell me about a time you took on work outside your comfort area.”*

Situation

At Slice, I was an SDE1 in a team with established infrastructure, deployment pipelines and senior engineers to learn from. When I co-founded StonksAI, all of that disappeared. I had to build the backend from scratch and run it in production, including areas I’d never owned.

Task

Learn enough, fast enough, to build and operate a reliable backend without a team to fall back on.

Action

I started by being honest about my gaps and listing them: [e.g., setting up infrastructure from zero, deployment and CI, cost management, security and auth design, operating production alone].

I learned each one by building something real, not just reading. For [infra], I [set up a small environment, broke it deliberately and learned how to recover]. For [cost], I [studied my own bill line by line]. I read other companies’ engineering blogs and postmortems to learn from mistakes I hadn’t made yet, and asked [experienced engineers I knew] specific questions when I was stuck.

I also treated my own mistakes as learning material. When something went wrong, I wrote down what happened and what I’d do differently.

Result

I built and ran the backend end to end. [What I can do now that I couldn’t at Slice, e.g., design, deploy, monitor and debug a system alone.] It also made me a better team engineer, because I now understand the parts other people used to handle for me.

Lesson

I learn fastest when there are real consequences. At Amazon, I want to keep stretching into areas I don’t know yet, like [a specific area relevant to the role].

Likely follow-ups

Q: What’s the hardest thing you had to learn?

[Real answer.]

Q: What are you learning right now?

[Have a current answer.]

6. Hire and Develop the Best

Story: **Mentoring a teammate (TEMPLATE)** · Source: Your choice

> *Sample question: “Tell me about a time you helped a team member grow.”*

This is a template: your resume has no mentoring example, so fill it with a real one. If you truly have none, say so honestly and describe your code-review approach and how you would mentor at SDE2.

Situation

At [Slice/StonksAI], [a junior engineer / intern / new teammate] joined and was struggling with [a specific gap, e.g., debugging production issues, writing testable code, understanding our services]. It was slowing their delivery and affecting their confidence.

Task

Help them become independent, not just unblock them each time. Doing the work for them would have fixed today’s problem and left the gap in place.

Action

I found the real gap by [watching how they approached a problem / asking them to walk me through their reasoning]. The issue was [e.g., they jumped to code changes before reproducing the bug].

I taught through real work. I paired with them on [a real task], explained why I did each step, then switched roles so they drove while I only asked questions.

I used code reviews to teach. Instead of just saying ‘change this,’ I explained the reasoning and pointed to examples, so the feedback applied beyond that one PR.

I gradually raised the difficulty, handing them [progressively harder tasks] and checking in regularly while stepping back over time.

Result

[Within X weeks/months, they were independently doing Y / shipped Z / were trusted to own a feature.] The team benefited too, because [less review back-and-forth / one more person who could handle on-call].

Lesson

Specific feedback with the reasoning attached works better than general advice. I also learned to step back earlier than felt comfortable.

7. Insist on the Highest Standards

Story: **Referral product** · Source: Slice

> *Sample question: “Tell me about a time you refused to compromise your standards.”*

Situation

At Slice, growth wanted a referral program, where existing users refer friends and both earn rewards. Nothing existed, so I built it from scratch. Referral programs involve money, so they attract abuse: fake accounts, reward farming and, more subtly, double payouts when a flaky mobile network makes the app retry a request.

Task

I owned the backend end to end, against a committed launch date [from growth/marketing]. The challenge was shipping on time without exposing the business to financial loss.

Action

Before writing code, I **listed the failure modes**: duplicate requests from retries, two concurrent requests both passing an eligibility check and both getting paid, fake accounts referring themselves, and partial failures where a reward is granted but the ledger entry fails. I reviewed this with [my lead] before building.

Then I built the safeguards in:

- **Idempotency:** each reward was keyed on [the referrer-referee pair plus the triggering event], enforced by a **unique constraint in the database**. Even with retries or two servers processing at once, a reward could only be created once. I deliberately didn’t rely on ‘check in code, then insert,’ which has a race window.
- **Concurrency:** I used [transactions / atomic updates / row-level locks] so simultaneous requests couldn’t both succeed.
- **Fraud checks:** [velocity limits per user and device, signals linking suspicious accounts, rewards gated on real activity].

Then came **pressure on the timeline**. These safeguards added about [X days]. I explained to [my manager / growth] that cutting them meant exposure to uncapped financial loss, not a normal shortcut, and offered to cut things that didn’t affect safety instead, [admin dashboard polish, extra reward tiers, some analytics], shipping them in a follow-up release.

Result

We launched on [date], and the program drove a **150% increase in user onboarding**, with [no double-payout incidents].

Lesson

When standards and speed conflict, I cut scope, not safety. Some shortcuts are cheap; ones involving money or trust almost never are.

Likely follow-ups

Q: Why a database constraint instead of checking in code?

Code checks have a gap between check and write. The constraint is atomic under any number of retries or servers.

Q: Did anything slip through?

[Real answer, plus how you fixed it.]

Q: How do you know referrals caused the 150%?

[Compared against the prior period / referral-attributed signups.] I treat it as directional.

> Backup story: the StonksAI quality bar, where I set tests, CI and logging as minimum standards even though nobody enforced them.

8. Think Big

Story: **Experimentation platform** · Source: Slice

> *Sample question: “Tell me about a time you saw an opportunity to do something much bigger than the initial focus.”*

Situation

At Slice, product decisions relied heavily on intuition, and launches were all-or-nothing. If a release had a problem, every user got it at once, and we usually couldn’t prove whether a change actually improved anything.

Task

The immediate need was [to test a specific change]. I saw that solving it as a one-off would mean rebuilding the same thing for every future test.

Action

So I built a **platform** instead of a single experiment:

- **Feature flags**, so behavior could be switched without a new deploy.
- **Staggered rollouts**, like 1% → 10% → 50% → 100%, so problems appeared on a small slice of users first.
- **Cohort-based deployments**, so changes could target specific user segments.

I designed it so other teams could run experiments themselves, without needing me or custom engineering. I also made assignment **consistent**: a user always saw the same variant, otherwise experiment results are meaningless.

Result

Teams could roll out gradually, roll back quickly and make decisions based on data instead of opinion. [Name one real experiment it enabled and the decision that came from it.]

Lesson

When I see the same problem recurring, I look for the platform that solves the whole category. It takes a little longer up front but multiplies everyone’s output.

Likely follow-ups

Q: How did you guarantee consistent assignment?
[Deterministic hashing of user ID per experiment.]

Q: How did you handle overlapping experiments?
[Real answer.]

Q: Was building a platform worth the extra time?
[Real numbers if you have them.]

9. Bias for Action

Story: **UPI Rewards** · Source: Slice

> *Sample question: “Give me an example of a calculated risk you took where speed was critical.”*

Situation

Slice wanted to grow user acquisition with UPI Rewards, gaming-style rewards shown after a user completes a UPI transaction. Timing mattered because [a competitor was moving / there was a campaign window].

Task

Build it quickly, but carefully, since it sat right next to the payments flow, where mistakes hurt users directly.

Action

I **shipped a minimal v1** focused only on [the core: triggering a reward after a successful transaction and showing it], rather than designing every feature upfront.

I **kept the risk contained.** The feature was [behind a flag / rolled out to a subset of users], and the reward logic was **decoupled from the payment itself**, so if rewards failed, the user’s payment still went through normally.

Where I lacked full information, I made **reversible decisions** and wrote down my assumptions, so we could correct course once real data came in. After launch, I **iterated** based on actual user behavior.

Result

It launched on time and increased user acquisition [add a real number]. Because we started small, [we caught X early and fixed it before full rollout].

Lesson

Speed and safety don’t conflict if you limit the blast radius. I ask: if this is wrong, how bad is it, and how fast can we undo it?

Likely follow-ups

Q: What did you deliberately leave out of v1?
[Real answer.]

Q: What risk did you accept, and did it happen?
[Real answer.]

10. Frugality

Story: **Bootstrapped backend** · Source: StonksAI

> *Sample question: “Tell me about a time you accomplished more with less.”*

Situation

StonksAI was bootstrapped. Every infrastructure dollar came out of our own runway, and there was no ops team. Over-provisioning or letting costs creep would directly shorten how long we could survive.

Task

Build a backend that was reliable and could grow, at minimal cost.

Action

I treated **cost as a design requirement**, not something to optimize later.

I chose between managed and self-managed based on **what was scarcest**. Where a managed service saved significant time, like [example], I paid for it, because my time was our most limited resource. Where cost dominated and operational burden was low, I [self-managed / chose a cheaper option].

I **deferred what we didn’t need yet**: [multi-region, a dedicated queueing system, large reserved capacity] weren’t justified by our traffic. I avoided building what I could reuse, [e.g., existing libraries/services for X].

I set up **cost monitoring and alerts**, so a runaway query or a loop bug couldn’t quietly burn money for a week.

Result

We ran at about [$X/month] while serving [N users]. [One concrete saving, e.g., ‘switching X saved Y% of our bill.’]

Lesson

Scale-ready, not scaled. Design so growth is possible, but only pay for the traffic you actually have.

Likely follow-ups

Q: What did you defer, and what was the risk?
[Real answer.]

Q: When would you have spent more?
[When X traffic level or reliability need justified it.]

> Backup story: MQTT at Slice, where cutting 90% of API calls delivered the same experience at a fraction of the cost.

11. Earn Trust

Story: **A bad technical decision** · Source: StonksAI

> *Sample question: “Tell me about a time you made a bad decision. What did you learn?”*

Situation

Early in building the StonksAI backend, I made [a specific wrong call, e.g., ‘a schema design that assumed X,’ ‘a library choice,’ ‘skipping tests on a critical path to move faster’]. At the time it seemed reasonable because [the reasoning].

Task

When it started causing problems, I needed to own it honestly with my co-founder, fix it without disrupting users and make sure I didn’t repeat it.

Action

I noticed it when [the symptom, e.g., queries slowed sharply as data grew / the same class of bug kept coming back / every feature in that area took twice as long].

I owned it immediately. I told my co-founder plainly: this was my decision, here’s what it’s costing us, and here’s my plan. No excuses, and no waiting until it got worse.

I fixed it carefully, [e.g., a staged migration: write to both old and new structures, backfill, verify, then switch reads], so users saw no downtime.

I built a mechanism so the lesson didn’t depend on my memory: [a short design-review checklist for irreversible decisions / tests required on critical paths / load tests before launch].

Result

The mistake cost us about [time/effort/impact]. After the fix, [improvement]. More importantly, the mechanism caught [a later similar risk] before it became a problem.

Lesson

Admitting mistakes early keeps their cost small and builds trust. A fix solves one problem; a mechanism prevents the next one.

Likely follow-ups

Q: With what you knew then, would you make the same call?

[Honest answer.]

Q: How did your co-founder react?

[Real answer.]

12. Dive Deep

Story: **Redis and p99 latency** · Source: Slice

> *Sample question: “Tell me about a situation where you had to dig deep to find the root cause.”*

Situation

At Slice, some core services had high p99 latency. Median response times looked fine, so most dashboards seemed healthy, but the slowest requests were slow enough that users felt delays on important screens. The database was also under heavy load.

Task

Find the real cause of the tail latency and fix it, without adding caching blindly, since stale data in a payments app causes real confusion.

Action

I deliberately **looked at percentiles, not averages.** Averages hid the problem; per-endpoint p99 showed exactly where the pain was.

I did **slow-query analysis** on those endpoints. The tail was driven by [a small number of read-heavy queries on data that rarely changed], repeatedly hitting the database.

Instead of caching everything, I **cached precisely those paths in Redis**, and spent the most time on **invalidation**: [TTLs plus explicit invalidation on writes] for data where staleness could mislead users, like [balances or statuses].

Finally, I **verified** with before-and-after metrics and kept those services on SLO dashboards so a regression would be visible.

Result

p99 latency dropped and database load fell substantially. [Real numbers, e.g., ‘p99 from X to Y ms, DB reads down Z%.’] Screens users had complained about became noticeably faster.

Lesson

Averages can hide real user pain, and diagnosing first means fixing the right thing. Cache invalidation is part of the design, not a cleanup task.

Likely follow-ups

Q: How did you handle cache stampedes?

[Real answer.]

Q: What happens if Redis goes down?

[Fall back to the DB, with safeguards against overloading it.]

13. Have Backbone; Disagree and Commit

Story: **Co-founder disagreement** · Source: StonksAI

> *Sample question: “Tell me about a time you strongly disagreed with a peer on something important.”*

Situation

At StonksAI, my co-founder wanted [e.g., to launch a feature by a date that meant skipping X / to take a technical shortcut]. I believed it would [hurt reliability / create technical debt that would block us within a few months].

Task

Make my case clearly without damaging a partnership the company depended on, and then deliver wholeheartedly, whatever we decided.

Action

I didn’t argue from instinct. I **prepared a case**: [an estimate of what the shortcut would cost us later, the specific risk, and evidence from X].

I **brought an alternative** rather than just a ‘no’: ‘Here’s a version that gets about 80% of the value in half the time, without the risk.’

I made sure I understood **why** they wanted it, [e.g., an investor demo / user demand], because that changed what a good solution looked like.

We [reached a compromise / they made the call]. Once decided, I **committed fully**. I didn’t slow-walk it or keep reopening the debate. I proposed a **checkpoint** to revisit it with real data in [X weeks].

Result

[What happened, and honestly, whether I was right.] Our working relationship got stronger, because we’d argued on the merits and then moved together.

Lesson

Disagree early, with data and an alternative, then commit completely. Being aligned and moving matters more than being proven right.

Likely follow-ups

Q: What if the decision had been clearly harmful?

I’d escalate with evidence, not just reluctance. That’s different from disagreeing on a judgment call.

Q: Did you turn out to be right?

[Be honest either way.]

> Use a real disagreement and fill in the specifics.

14. Deliver Results

Story: **Hard launch deadline** · Source: StonksAI

> *Sample question: “Tell me about a time you delivered an important project under a tight deadline.”*

Situation

At StonksAI, we had a hard date for [launch / an investor demo / an app store release], and the backend wasn’t finished. Missing it meant [lost momentum / a missed opportunity / a bad first impression].

Task

Ship on time, with the critical paths solid. Shipping something broken would have been worse than slipping.

Action

I re-scoped ruthlessly. I listed everything remaining and split it into must-have for day one and can ship in a patch. The test: will a user notice it’s missing on day one, and can it break anything important?

I protected the critical paths. [Data correctness, auth, the core user flow] got full testing; lower-risk parts got lighter treatment, deliberately.

I made decisions quickly. Where waiting for perfect information would cost days, I made reversible calls and wrote down the risks I was accepting, like [X].

I communicated daily with my co-founder: what was done, what was at risk and what I’d cut, so there were no surprises.

Result

We launched on [date]. [What shipped, what slipped, and how quickly the follow-up patch came.] [Any early result: first users, demo outcome.]

Lesson

A deadline is a prioritization tool. I cut scope, not quality on critical paths, and I make the trade-offs visible.

Likely follow-ups

Q: What did you cut, and did users notice?
[Real answer.]

Q: Did any accepted risk come true?
[Real answer.]

> Backup story: the referral launch at Slice. Use it here only if you didn’t use it for Insist on the Highest Standards.

15. Strive to Be Earth’s Best Employer

Story: **Supportive team environment (TEMPLATE)** · Source: Your choice

> *Sample question: “Tell me about a time you helped build a more supportive or inclusive working environment.”*

> This is a template: your resume has no team-culture example, so fill it with a real one.

Situation

At [Slice/StonksAI], I noticed [e.g., new joiners took weeks to become productive because knowledge lived in people’s heads / quieter teammates rarely spoke up in design discussions / people hid mistakes because incidents felt like blame sessions].

Task

Make it easier for everyone to contribute and succeed, not just the people who were already comfortable.

Action

I [wrote onboarding docs for our services / set up a buddy system / explicitly asked quieter teammates for input in design reviews / gave public credit for others’ work].

After incidents, I **pushed for blameless post-mortems** focused on what in the system allowed the failure rather than who caused it, so people felt safe raising mistakes early.

[Any other concrete action.]

Result

[New joiners were productive in X instead of Y / more people contributed to design discussions / mistakes were reported earlier.]

Lesson

A good environment is mostly built from small, consistent habits, not big programs.

16. Success and Scale Bring Broad Responsibility

Story: **Referral fraud and honest users** · Source: Slice

> *Sample question: “Tell me about a time you considered the impact of your work beyond your direct customers.”*

Situation

When I built Slice’s referral program, the obvious goal was growth. But a referral program affects more people than the user who signs up: the referrer, the friend being invited, honest users competing for the same incentives, and the business funding the rewards.

Task

Make it hard to abuse, while making sure the protections didn’t hurt honest users.

Action

I built fraud checks, [velocity limits, linked-account signals, activity requirements], to stop reward farming, which would have drained a budget meant for real users.

I also thought about the **side effects of the fraud checks themselves.** Overly aggressive rules would block legitimate users, like [family members on the same device, or users with new phones]. So I [tuned thresholds conservatively, reviewed flagged cases and gave blocked users a path to resolution] instead of silently rejecting them.

I made sure invites [respected the referred person, e.g., not spamming contacts], since those people never chose to interact with us.

Result

The program grew onboarding by **150%** while [keeping fraud low and false positives rare]. Honest users weren’t collateral damage.

Lesson

At scale, every rule I write affects people I never see. I think about who else is touched, not just the primary user.

> This reuses the referral project from a different angle. If you already told the referral story in the same loop, use this alternative: polling every few seconds drained battery and mobile data for every user, hurting most on low-end phones and limited data plans. Cutting calls by 90% with MQTT wasn’t just a server saving; it removed a cost we were silently imposing on users.

Part 2: Story Bank by Company

Every story in full STAR form, grouped by Slice and StonksAI, with the principles each one can answer.

Story bank index

Code	Story	Best for
S1	Referral incentive product	Ownership, Deliver Results, Insist on the Highest Standards, Dive Deep, Broad Responsibility
S2	MQTT replacing polling	Invent and Simplify, Frugality, Are Right A Lot, Dive Deep
S3	Redis caching and p99 latency	Dive Deep, Deliver Results, Customer Obsession
S4	NPS platform (rating 3.3 → 4.5)	Customer Obsession, Ownership, Deliver Results
S5	SLO dashboards and alerting	Insist on the Highest Standards, Ownership, Dive Deep
S6	Experimentation platform	Think Big, Invent and Simplify, Learn and Be Curious
S7	UPI Rewards	Bias for Action, Deliver Results
K1	Choosing the architecture with no data	Are Right A Lot, Think Big, Bias for Action, Invent and Simplify
K2	Owning the production outage	Ownership, Dive Deep, Earn Trust
K3	Building on a bootstrapped budget	Frugality, Invent and Simplify, Ownership
K4	Disagreeing with my co-founder	Have Backbone; Disagree and Commit, Earn Trust
K5	A bad technical decision	Are Right A Lot, Earn Trust, Learn and Be Curious
K6	Setting the quality bar with nobody checking	Insist on the Highest Standards, Dive Deep
K7	User feedback shaping the backend	Customer Obsession, Learn and Be Curious
K8	Hitting a hard launch deadline	Deliver Results, Bias for Action

S = Slice stories (resume facts). K = StonksAI stories (based only on building the backend 0 → 1 as co-founder). Several stories also appear in Part 1; here they are grouped by company so you can see your full inventory at a glance.

SLICE STORIES

S1. Referral incentive product

> **Best for:** Ownership, Deliver Results, Insist on the Highest Standards, Dive Deep, Broad Responsibility

Situation

At Slice, growth wanted a referral program, where existing users refer friends and both earn rewards. Nothing existed, so I built it from scratch. Referral programs involve money, so they attract abuse: fake accounts, reward farming and, more subtly, double payouts when a flaky mobile network makes the app retry a request.

Task

I owned the backend end to end, against a committed launch date [from growth/marketing]. The challenge was shipping on time without exposing the business to financial loss.

Action

Before writing code, I **listed the failure modes**: duplicate requests from retries, two concurrent requests both passing an eligibility check and both getting paid, fake accounts referring themselves, and partial failures where a reward is granted but the ledger entry fails. I reviewed this with [my lead] before building.

Then I built the safeguards in:

- **Idempotency:** each reward was keyed on [the referrer-referee pair plus the triggering event], enforced by a **unique constraint in the database**. Even with retries or two servers processing at once, a reward could only be created once. I deliberately didn't rely on 'check in code, then insert,' which has a race window.
- **Concurrency:** I used [transactions / atomic updates / row-level locks] so simultaneous requests couldn't both succeed.
- **Fraud checks:** [velocity limits per user and device, signals linking suspicious accounts, rewards gated on real activity].

Then came **pressure on the timeline**. These safeguards added about [X days]. I explained to [my manager / growth] that cutting them meant exposure to uncapped financial loss, not a normal shortcut, and offered to cut things that didn't affect safety instead, [admin dashboard polish, extra reward tiers, some analytics], shipping them in a follow-up release.

Result

We launched on [date], and the program drove a **150% increase in user onboarding**, with [no double-payout incidents].

Lesson

When standards and speed conflict, I cut scope, not safety. Some shortcuts are cheap; ones involving money or trust almost never are.

Likely follow-ups

Q: Why a database constraint instead of checking in code?

Code checks have a gap between check and write. The constraint is atomic under any number of retries or servers.

Q: Did anything slip through?

[Real answer, plus how you fixed it.]

Q: How do you know referrals caused the 150%?

[Compared against the prior period / referral-attributed signups.] I treat it as directional.

S2. MQTT replacing polling

> **Best for:** Invent and Simplify, Frugality, Are Right A Lot, Dive Deep

Situation

Our Android app at Slice repeatedly polled backend APIs to check whether something had changed, such as [transaction or reward status]. When I looked at the traffic, the large majority of those requests returned ‘no change.’ We were spending server capacity on empty calls, and users were spending battery and mobile data on them.

Task

Cut that wasted load without making the app feel staler, and ideally make it more real-time.

Action

I started by **measuring**, confirming from request logs how much polling traffic was redundant, so I knew the size of the win before changing anything.

Then I **compared options**:

- Longer polling intervals were the simplest, but would make data staler.
- WebSockets would work, but persistent connections at mobile scale are heavier on servers and device battery.
- MQTT is a lightweight publish-subscribe protocol built for unreliable networks, which fits mobile users.

I chose MQTT. The server publishes only when state actually changes, and the client subscribes. ‘Keep asking’ became ‘get told when something happens.’

I **managed the risk**: gradual rollout behind a feature flag with polling as fallback, so a broker problem wouldn’t break the feature, plus reconnect handling so a phone that went offline fetched any missed state when it came back.

Result

API calls dropped by about **90%**, and server load dropped with them. Users got updates in real time instead of waiting for the next poll. The simpler model was both cheaper and better.

Lesson

The simplest fix isn’t always the first one you think of. Changing the communication model removed the problem instead of tuning around it.

Likely follow-ups

Q: Why not WebSockets or server-sent events?

Heavier connection management on mobile. MQTT is designed for flaky networks and low overhead.

Q: What if a client misses a message?

On reconnect, the client fetched current state, so correctness never depended on every message arriving.

Q: How did you measure 90%?

[Calls per active user, same time window, before vs. after rollout.]

S3. Redis caching and p99 latency

> **Best for:** Dive Deep, Deliver Results, Customer Obsession

Situation

At Slice, some core services had high p99 latency. Median response times looked fine, so most dashboards seemed healthy, but the slowest requests were slow enough that users felt delays on important screens. The database was also under heavy load.

Task

Find the real cause of the tail latency and fix it, without adding caching blindly, since stale data in a payments app causes real confusion.

Action

I deliberately **looked at percentiles, not averages.** Averages hid the problem; per-endpoint p99 showed exactly where the pain was.

I did **slow-query analysis** on those endpoints. The tail was driven by [a small number of read-heavy queries on data that rarely changed], repeatedly hitting the database.

Instead of caching everything, I **cached precisely those paths in Redis**, and spent the most time on **invalidation**: [TTLs plus explicit invalidation on writes] for data where staleness could mislead users, like [balances or statuses].

Finally, I **verified** with before-and-after metrics and kept those services on SLO dashboards so a regression would be visible.

Result

p99 latency dropped and database load fell substantially. [Real numbers, e.g., 'p99 from X to Y ms, DB reads down Z%.'] Screens users had complained about became noticeably faster.

Lesson

Averages can hide real user pain, and diagnosing first means fixing the right thing. Cache invalidation is part of the design, not a cleanup task.

Likely follow-ups

Q: How did you handle cache stampedes?
[Real answer.]

Q: What happens if Redis goes down?
[Fall back to the DB, with safeguards against overloading it.]

S4. NPS platform (rating 3.3 → 4.5)

> **Best for:** Customer Obsession, Ownership, Deliver Results

Situation

At Slice, our Google Play rating was 3.3. For a fintech app, that's a real problem, because people decide whether to trust you with their money partly from that number. We also didn't know *why* it was low. Feedback came in through scattered channels like support tickets, random reviews and social media, and the team was mostly guessing what users disliked.

Task

I wanted to replace guessing with a reliable system: hear from customers at the right moment, understand what was hurting them, and turn that into fixes. I owned it across both the backend and the Android app.

Action

I built an end-to-end NPS platform, and I spent most of my design effort on three things.

Asking at the right moment. A survey that interrupts someone mid-payment is itself a bad experience. So the backend decided who to ask and when, [after a completed transaction, not too often, with a cooldown], so we got honest answers without annoying people.

Routing responses so we could act. Users who rated us highly were nudged toward leaving a Play Store review. Users who were unhappy were routed to in-app feedback, where their issue could actually reach us and be resolved. An unhappy user posting a 1-star review gets no help, while one who reaches us can.

Closing the loop. I aggregated responses into themes, [e.g., confusion in a specific flow, slow screens, a recurring bug], and brought them to the team so priorities followed real customer pain instead of assumptions.

Result

The Play Store rating rose from **3.3 to 4.5**. Part came from asking happy users at better moments, and part came from the fixes the feedback revealed. Both mattered.

Lesson

Collecting feedback isn't customer obsession. Acting on it is. Today, I'd also track how quickly feedback themes turn into shipped fixes.

Likely follow-ups

Q: Isn't routing unhappy users away from the Play Store manipulative?
Nobody was blocked from reviewing. We gave unhappy users a faster path to an actual fix, and the rating rose partly because we fixed what they reported.

Q: How did you avoid survey fatigue?
[Frequency caps and cooldowns per user.]

Q: What was the most surprising thing customers said?
[Have one real example ready.]

S5. SLO dashboards and alerting

> **Best for:** Insist on the Highest Standards, Ownership, Dive Deep

Situation

At Slice, our critical services had no shared definition of ‘healthy.’ We didn’t have agreed latency or availability targets, and alerting was patchy. Too often, we found out about problems from support tickets or users before our own systems told us. For a payments product, that’s a trust problem.

Task

Reliability tooling wasn’t assigned to me; my job was building product features. But I was regularly debugging issues that monitoring should have caught earlier, and nobody owned this gap, so I took it.

Action

Defining what good means. For each critical service, I defined SLOs for latency, availability and error rates, based on [historical data and what users actually experienced], not arbitrary numbers.

Making it visible. I built Grafana dashboards around those SLOs, so anyone on the team could see a service’s health at a glance.

Alerting people would trust. If alerts fire constantly, people ignore them; if thresholds are too loose, you miss real incidents. I tuned thresholds against historical data and made sure each alert pointed to a specific action.

I did this alongside my feature work, and walked the team through the dashboards so it became shared, not just mine.

Result

The team began catching incidents faster, often before users noticed, and reliability improved. [Give one concrete example: an alert caught X before it reached users.] Conversations changed from ‘it feels slow today’ to ‘we’re burning our latency budget on this endpoint,’ which made prioritization easier.

Lesson

If I see a gap that hurts customers and nobody owns it, I treat it as mine. Next time I’d bring the team in earlier, so ownership spreads faster.

Likely follow-ups

Q: How did you pick the SLO targets?
[Historical percentiles plus user impact.]

Q: How did you avoid alert fatigue?
[Actionable alerts only, tuned thresholds, regular review of noisy alerts.]

Q: How did your manager react to you doing this outside your scope?
[Real answer.]

S6. Experimentation platform

> **Best for:** Think Big, Invent and Simplify, Learn and Be Curious

Situation

At Slice, product decisions relied heavily on intuition, and launches were all-or-nothing. If a release had a problem, every user got it at once, and we usually couldn't prove whether a change actually improved anything.

Task

The immediate need was [to test a specific change]. I saw that solving it as a one-off would mean rebuilding the same thing for every future test.

Action

So I built a **platform** instead of a single experiment:

- **Feature flags**, so behavior could be switched without a new deploy.
- **Staggered rollouts**, like 1% → 10% → 50% → 100%, so problems appeared on a small slice of users first.
- **Cohort-based deployments**, so changes could target specific user segments.

I designed it so other teams could run experiments themselves, without needing me or custom engineering. I also made assignment **consistent**: a user always saw the same variant, otherwise experiment results are meaningless.

Result

Teams could roll out gradually, roll back quickly and make decisions based on data instead of opinion. [Name one real experiment it enabled and the decision that came from it.]

Lesson

When I see the same problem recurring, I look for the platform that solves the whole category. It takes a little longer up front but multiplies everyone's output.

Likely follow-ups

Q: How did you guarantee consistent assignment?
[Deterministic hashing of user ID per experiment.]

Q: How did you handle overlapping experiments?
[Real answer.]

Q: Was building a platform worth the extra time?
[Real numbers if you have them.]

S7. UPI Rewards

> **Best for:** Bias for Action, Deliver Results

Situation

Slice wanted to grow user acquisition with UPI Rewards, gaming-style rewards shown after a user completes a UPI transaction. Timing mattered because [a competitor was moving / there was a campaign window].

Task

Build it quickly, but carefully, since it sat right next to the payments flow, where mistakes hurt users directly.

Action

I **shipped a minimal v1** focused only on [the core: triggering a reward after a successful transaction and showing it], rather than designing every feature upfront.

I **kept the risk contained.** The feature was [behind a flag / rolled out to a subset of users], and the reward logic was **decoupled from the payment itself**, so if rewards failed, the user's payment still went through normally.

Where I lacked full information, I made **reversible decisions** and wrote down my assumptions, so we could correct course once real data came in. After launch, I **iterated** based on actual user behavior.

Result

It launched on time and increased user acquisition [add a real number]. Because we started small, [we caught X early and fixed it before full rollout].

Lesson

Speed and safety don't conflict if you limit the blast radius. I ask: if this is wrong, how bad is it, and how fast can we undo it?

Likely follow-ups

Q: What did you deliberately leave out of v1?
[Real answer.]

Q: What risk did you accept, and did it happen?
[Real answer.]

STONKSAI STORIES

K1. Choosing the architecture with no data

> **Best for:** Are Right A Lot, Think Big, Bias for Action, Invent and Simplify

Situation

When I co-founded StonksAI, I built the backend from scratch, starting with an empty repository. We had no users, no traffic data and no benchmarks. Every architecture decision was a bet, and as the person building the backend, they were mine to make.

Task

Choose an architecture that let us ship quickly without boxing us into a rewrite once real users arrived.

Action

I separated reversible decisions from irreversible ones. Choosing a framework or organizing internal code is cheap to change later. The data model, public API contracts and authentication are expensive once users and clients depend on them. So I moved fast on the cheap decisions and spent real design time on the expensive ones.

I chose the operating model for a tiny team. I compared [microservices with a modular monolith]. Microservices give independent scaling, but bring deployment, networking and monitoring overhead that would have consumed my time. I chose [a modular monolith] with clean internal boundaries, so splitting out a service later would be a refactor, not a rewrite.

I tested the riskiest assumption first, [e.g., whether the core flow could handle realistic load], with a quick prototype before committing.

I sought disagreement. I walked [experienced engineers / mentors] through the design and asked them to find holes. [One thing they changed my mind on.]

Result

We shipped the first version in [X weeks]. [What held up, e.g., ‘the core data model never needed a painful migration.’] Not everything was right: [one thing I’d change, e.g., ‘I drew one module boundary in the wrong place and had to refactor it.’]

Lesson

When data is missing, I make the irreversible decisions carefully and keep everything else cheap to change. Being right a lot partly means being easy to correct when you’re wrong.

Likely follow-ups

Q: What would make you split the monolith?

[A component with very different scaling needs, or team growth.]

Q: What did you get wrong?

[Have a real answer.]

K2. Owning the production outage

> **Best for:** Ownership, Dive Deep, Earn Trust

Situation

With a founding team that small, I was backend, DevOps and on-call at StonksAI. One [evening/night], [the API became slow/unavailable] in production, and nobody else could fix it.

Task

Restore service quickly, then make sure it couldn’t happen the same way again.

Action

First I **stabilized**: I [rolled back / scaled / disabled the faulty path] to restore service, and updated my co-founder and [affected users] so nobody was guessing.

Then I **dug for the real cause**. I looked at [logs, metrics, query plans] and ruled out the obvious suspect, [X], because [the data showed Y]. The actual root cause was [Z].

Finally I wrote a **no-blame post-mortem** and added [monitoring, an alert, a test, a runbook] so that class of failure would be caught earlier.

Result

Service was restored in [X minutes], and the issue never recurred. A [later similar issue] was caught by the new alert before users noticed.

Lesson

Fix the symptom first, then the cause, then the system that let it happen.

Likely follow-ups

Q: How did you decide between rollback and fix-forward?
[Real answer: rollback when the cause is unclear and a known-good version exists.]

Q: What would you do differently with a larger team?
[Real answer.]

K3. Building on a bootstrapped budget

> **Best for:** Frugality, Invent and Simplify, Ownership

Situation

StonksAI was bootstrapped. Every infrastructure dollar came out of our own runway, and there was no ops team. Over-provisioning or letting costs creep would directly shorten how long we could survive.

Task

Build a backend that was reliable and could grow, at minimal cost.

Action

I treated **cost as a design requirement**, not something to optimize later.

I chose between managed and self-managed based on **what was scarcest**. Where a managed service saved significant time, like [example], I paid for it, because my time was our most limited resource. Where cost dominated and operational burden was low, I [self-managed / chose a cheaper option].

I **deferred what we didn't need yet**: [multi-region, a dedicated queueing system, large reserved capacity] weren't justified by our traffic. I avoided building what I could reuse, [e.g., existing libraries/services for X].

I set up **cost monitoring and alerts**, so a runaway query or a loop bug couldn't quietly burn money for a week.

Result

We ran at about [$X/month] while serving [N users]. [One concrete saving, e.g., 'switching X saved Y% of our bill.']

Lesson

Scale-ready, not scaled. Design so growth is possible, but only pay for the traffic you actually have.

Likely follow-ups

Q: What did you defer, and what was the risk?
[Real answer.]

Q: When would you have spent more?
[When X traffic level or reliability need justified it.]

K4. Disagreeing with my co-founder

> **Best for:** Have Backbone; Disagree and Commit, Earn Trust

Situation

At StonksAI, my co-founder wanted [e.g., to launch a feature by a date that meant skipping X / to take a technical shortcut]. I believed it would [hurt reliability / create technical debt that would block us within a few months].

Task

Make my case clearly without damaging a partnership the company depended on, and then deliver wholeheartedly, whatever we decided.

Action

I didn't argue from instinct. I **prepared a case**: [an estimate of what the shortcut would cost us later, the specific risk, and evidence from X].

I **brought an alternative** rather than just a 'no': 'Here's a version that gets about 80% of the value in half the time, without the risk.'

I made sure I understood **why** they wanted it, [e.g., an investor demo / user demand], because that changed what a good solution looked like.

We [reached a compromise / they made the call]. Once decided, I **committed fully**. I didn't slow-walk it or keep reopening the debate. I proposed a **checkpoint** to revisit it with real data in [X weeks].

Result

[What happened, and honestly, whether I was right.] Our working relationship got stronger, because we'd argued on the merits and then moved together.

Lesson

Disagree early, with data and an alternative, then commit completely. Being aligned and moving matters more than being proven right.

Likely follow-ups

Q: What if the decision had been clearly harmful?

I'd escalate with evidence, not just reluctance. That's different from disagreeing on a judgment call.

Q: Did you turn out to be right?

[Be honest either way.]

K5. A bad technical decision

> **Best for:** Are Right A Lot, Earn Trust, Learn and Be Curious

Situation

Early in building the StonksAI backend, I made [a specific wrong call, e.g., ‘a schema design that assumed X,’ ‘a library choice,’ ‘skipping tests on a critical path to move faster’]. At the time it seemed reasonable because [the reasoning].

Task

When it started causing problems, I needed to own it honestly with my co-founder, fix it without disrupting users and make sure I didn’t repeat it.

Action

I noticed it when [the symptom, e.g., queries slowed sharply as data grew / the same class of bug kept coming back / every feature in that area took twice as long].

I owned it immediately. I told my co-founder plainly: this was my decision, here’s what it’s costing us, and here’s my plan. No excuses, and no waiting until it got worse.

I fixed it carefully, [e.g., a staged migration: write to both old and new structures, backfill, verify, then switch reads], so users saw no downtime.

I built a mechanism so the lesson didn’t depend on my memory: [a short design-review checklist for irreversible decisions / tests required on critical paths / load tests before launch].

Result

The mistake cost us about [time/effort/impact]. After the fix, [improvement]. More importantly, the mechanism caught [a later similar risk] before it became a problem.

Lesson

Admitting mistakes early keeps their cost small and builds trust. A fix solves one problem; a mechanism prevents the next one.

Likely follow-ups

Q: With what you knew then, would you make the same call?

[Honest answer.]

Q: How did your co-founder react?

[Real answer.]

K6. Setting the quality bar with nobody checking

> **Best for:** Insist on the Highest Standards, Dive Deep

Situation

In a startup, nothing forces quality. At StonksAI we could have shipped faster by skipping tests, logging and monitoring, and no one would have stopped me.

Task

Set a bar that protected users and our ability to move fast later, without slowing us to a crawl.

Action

I set a **minimum standard** for the backend: [automated tests on core flows, CI before deploy, structured logging, error tracking].

I was explicit about the trade-off: some corners are fine to cut, but never [data correctness, auth, or anything affecting user trust]. I'd rather slip a feature a couple of days than ship something I'd have to explain to users.

Result

Deploys went from nervous to routine, with [fewer regressions and faster debugging].

Lesson

A quality bar is a speed investment. It pays back on every deploy.

Likely follow-ups

Q: What did you skip, and why was that acceptable?
[Real answer.]

Q: How would you scale this with more engineers?
[e.g., required reviews, CI gates, shared standards doc.]

K7. User feedback shaping the backend

> **Best for:** Customer Obsession, Learn and Be Curious

Situation

After launching StonksAI, early users told us what felt [slow, broken or missing].

Task

Decide which feedback to act on, since I couldn’t build everything.

Action

I talked to users directly and compared **what they said with what the usage data showed**, since those often differ.

I prioritized [the fix] because it hit the core experience for the most users, and said no to [a request that didn’t fit].

I worked backwards from the experience I wanted, [‘results should feel instant’], and let that drive backend choices like [caching, precomputation or API shape].

Result

[Latency / retention / complaints / ratings] improved after the change.

Lesson

Customer obsession includes saying no, because focus serves customers better than chasing every request.

Likely follow-ups

Q: How did you decide what to say no to?
[Real answer.]

Q: What did users say that surprised you?
[Real answer.]

K8. Hitting a hard launch deadline

> **Best for:** Deliver Results, Bias for Action

Situation

At StonksAI, we had a hard date for [launch / an investor demo / an app store release], and the backend wasn't finished. Missing it meant [lost momentum / a missed opportunity / a bad first impression].

Task

Ship on time, with the critical paths solid. Shipping something broken would have been worse than slipping.

Action

I re-scoped ruthlessly. I listed everything remaining and split it into must-have for day one and can ship in a patch. The test: will a user notice it's missing on day one, and can it break anything important?

I protected the critical paths. [Data correctness, auth, the core user flow] got full testing; lower-risk parts got lighter treatment, deliberately.

I made decisions quickly. Where waiting for perfect information would cost days, I made reversible calls and wrote down the risks I was accepting, like [X].

I communicated daily with my co-founder: what was done, what was at risk and what I'd cut, so there were no surprises.

Result

We launched on [date]. [What shipped, what slipped, and how quickly the follow-up patch came.] [Any early result: first users, demo outcome.]

Lesson

A deadline is a prioritization tool. I cut scope, not quality on critical paths, and I make the trade-offs visible.

Likely follow-ups

Q: What did you cut, and did users notice?
[Real answer.]

Q: Did any accepted risk come true?
[Real answer.]

Final checklist before the interview

• **Fill every bracket.** Expect “what exactly was the root cause?” or “what was the p99 before?” Invented specifics fall apart quickly.

• **Find real examples for Hire and Develop the Best and Strive to Be Earth’s Best Employer.** These are your biggest gaps, and SDE2 loops usually include at least one people-oriented question.

• **Have one number per story** and know how it was measured: 150% onboarding, 90% fewer API calls, rating 3.3 → 4.5.

• **End every story with a lesson** and, ideally, what you would do differently.

• **Practice out loud with a timer.** About 2 minutes per first telling.

• **Say “I” for your decisions**, especially for Slice stories where you worked within a team.

• **Spread stories across interviewers** using the story map; never repeat one in the same loop.